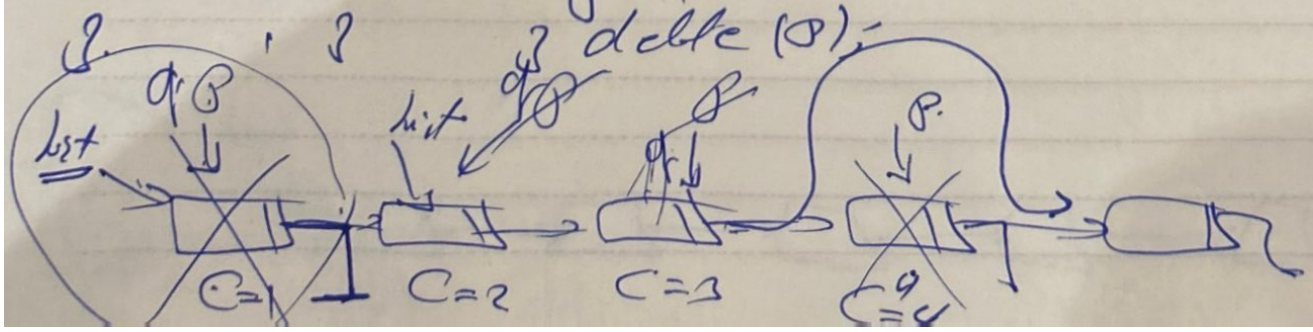
$(x == 3)$?
$x = 3$

18/3/2025

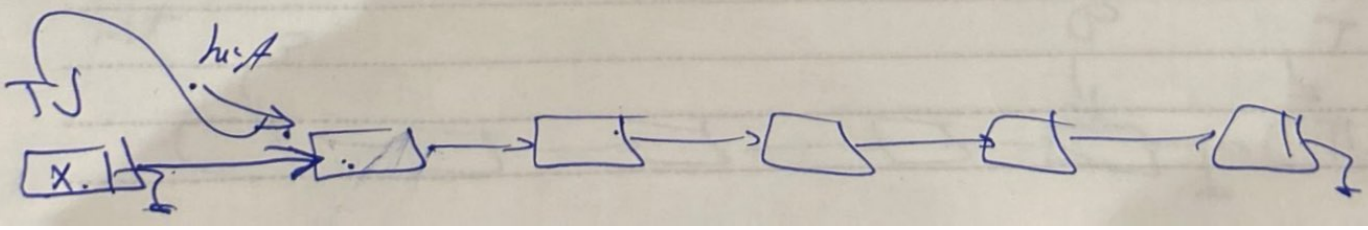
```
void deleteKth (node * list, int K)
{
        if( list != NULL)
        {   Node *p = list;
            if( K == 1)
            {  list = list->next;
               p -> Next = NULL;
               Delete (p);
            }
            else
               int C = 1;

               Node * q;
               while (p != NULL) && (C < K)
               {
                 q = p;
               p = p -> Next
                 C++;
               }
               if(p != NULL)
                { q -> Next = p -> Next;
                  p -> Next = NULL;
                  delete (p);
```



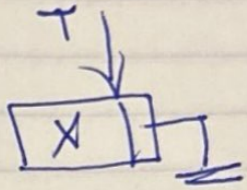
```
void Insertfirst( Node * list, int x)
{
    Node * T;
    T = new Node();
    T->Data = x;
    T->Next = NULL;

    T->Next = list;
    list = T;
}
```

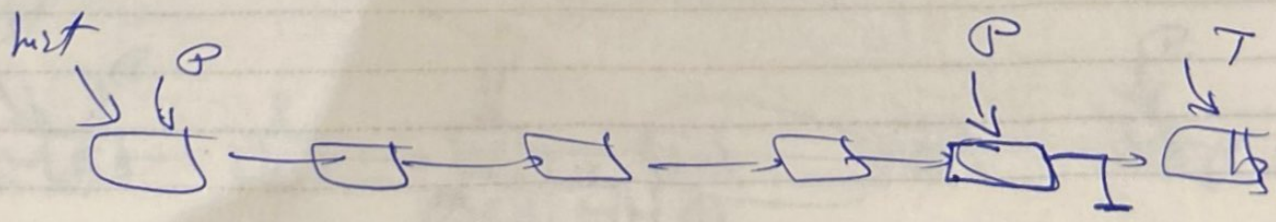
Node Creation

```
if (list == NULL)
    list = T;
else
{ T->Next = list
  list = T;
}
```

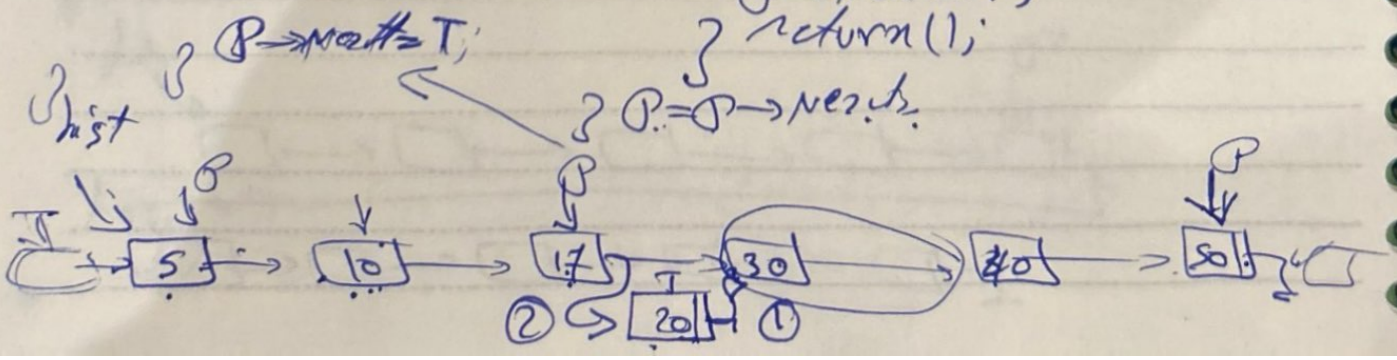


```
void InsertLast (node *list, int x)
{
    Node *T = new Node();
    T->Data = x;
    T->Next = NULL;
    if(list == NULL)
        list = T;
    else
    {
        Node *P = list;
        while (P->Next != NULL)
            P = P->Next;
        P->Next = T;
    }
}
```

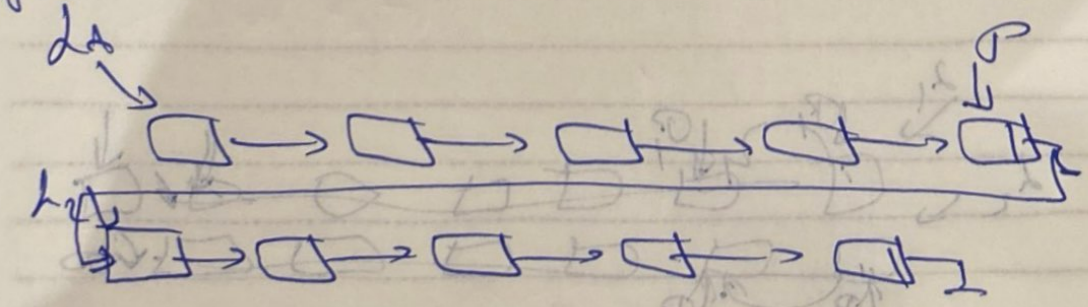




```
void InsertSorted (Node* list, int x)
{
    Node* T = new Node();
    T->Data = x;
    T->Next = NULL;
    if(list == NULL)
        list = T;
    else
    {
        if(x < list->Data)
        {
            T->Next = list;
            list = T;
        }
        else
        {
            Node* P = list;
            while(P->Next != NULL)
            {
                ~~P = P->Next~~
                if(P->Next->Data > x)
                {
                    T->Next = P->Next;
                    P->Next = T;
                    return(1);
                }
                P = P->Next;
            }
            P->Next = T;
```



```
void Concatination(Node* h1, Node* h2)
{
    if(h1 == NULL)
        h1 = h2;
    else
    {   Node *P = h1;
        while (P->Next != NULL)
            P = P->Next;

        P->Next = h2;
    }
}
```

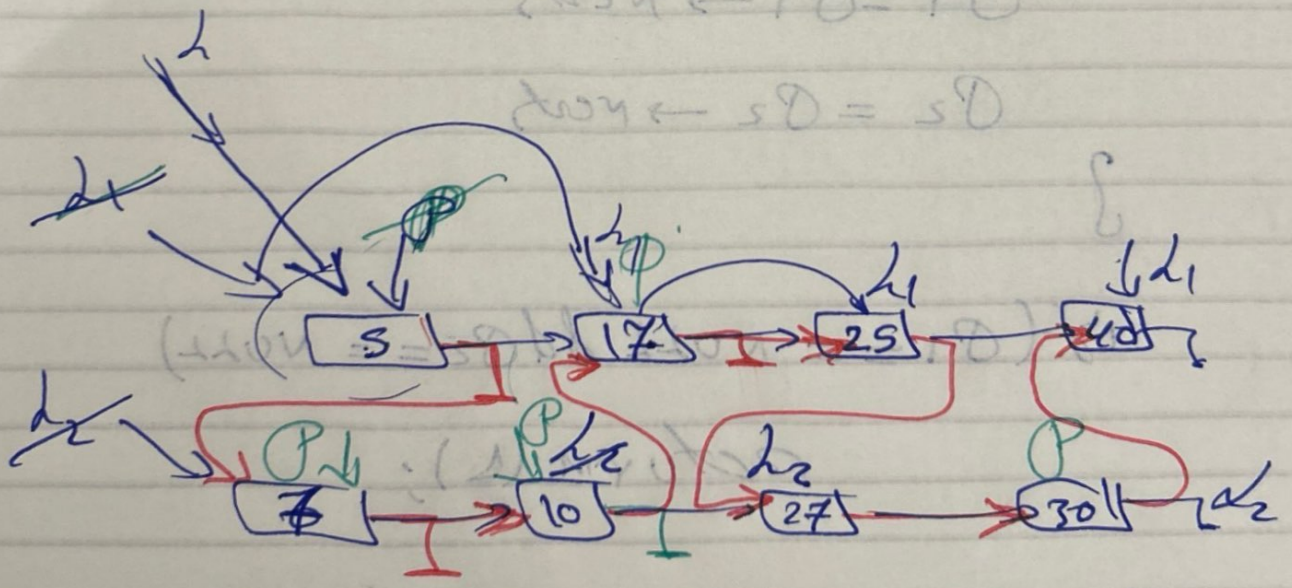


```
int Identical( Node * h1 , Node*h2)
{
        Node * P1 = h1;
        Node * P2 = h2;
        while (P1 != NULL && P2 != NULL)
        {
            if(P1 -> Data != P2 -> Data)
                return(0);

            P1 = P1 -> Next;
            P2 = P2 -> Next;
        }
        if(P1 == NULL && P2 == NULL)
            return (1);

    return(0);
}
```

combine

```
void union (Node *h1, Node *h2, Node *h)
{
    while (h1 != NULL && h2 != NULL)
    {
        if(h1->data < h2->data)
        {   if(h == NULL)
            {   h = h1;
                h1 = h1 -> next;
                Node *p = h;
                p -> next = NULL;
            }
            else{   p -> next = h1;
                    p = h1;
                    h1 = h1 -> next;
                    p -> next = NULL;
            }
        }
```

```
else{ if(L == NULL)
        { L = L2;
          L2 = L2 -> Next;
          Node *P = L;
          P -> Next = NULL;
        }
      else{ P -> Next = L2;
            P = L2;
            L2 = L2 -> Next;
            P -> Next = NULL;
      }
    }
}//while
if( L1 != NULL)
    P -> Next = L1;
if (L2 != NULL)
    P -> Next = L2;
}
```


5
17
25
40
7
10
27
30